Modül 18

Component Communication

Parent-Child, Sibling, Cross-Tree

Modül:	18 / 30
Ders Sayısı:	7 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 18'e Hoş Geldin

Component'ler arasında veri akışı — Angular mimarisinin kalbi. Modern Angular'da signal-based `input()`/`output()`, two-way binding, content projection, ve cross-tree communication pattern'lerini öğreneceğiz.

Öğrenecekler:

- ✓ input() ve output() signal API
- ✓ Two-way binding — model()
- ✓ Content projection (ng-content)
- ✓ Parent → Child: ViewChild / contentChild
- ✓ Child → Parent: output() event emitter
- ✓ Sibling: shared service pattern
- ✓ Cross-tree: global state service

DERS 18.1

input() — Modern Property Binding

Parent'tan child'a veri aktarmanın modern yolu.

input() Syntax

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-user-card',
  template: `
    <div>
      <h3>{{ user().name }}</h3>
      <p>{{ user().email }}</p>
      @if (showActions()) {
        <button>Edit</button>
      }
    </div>
  `
})
export class UserCardComponent {
  // Required input
  user = input.required<User>();

  // Optional with default
  showActions = input(true);

  // Optional with explicit type
  highlight = input<string | undefined>(undefined);

  // Alias
  cssClass = input<string>('', { alias: 'class' });

  // Transform
  count = input(0, { transform: (v: string | number) => +v });
}
```

Parent'tan Kullanım

```
<app-user-card
  [user]="currentUser()"
  [showActions]="isAdmin()">
</app-user-card>
```

⚡ input() vs @Input()

Konu	@Input() (Eski)	input() (Modern)
Type	Property	Signal
Reactivity	Manuel (ngOnChanges)	Otomatik
Required	! tehlikeli	input.required<T>()
Transform	@Input({ transform })	transform option
Default	Class field =	input(defaultValue)

DERS 18.2

output() — Modern Event Emitter

Child'tan parent'a event göndermek.

output() Syntax

```
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-button',
  template: `
    <button (click)="onClick()">Click me</button>
  `
})
export class ButtonComponent {
  clicked = output<void>();
  saved = output<{ id: string; data: any }>();
  cancelled = output();

  onClick() {
    this.clicked.emit();
  }

  save() {
    this.saved.emit({ id: '123', data: { x: 1 } });
  }
}
```

Parent'ta Listen

```
<app-button
  (clicked)="onButtonClicked()"
  (saved)="onSaved($event)">
</app-button>
```

outputFromObservable

RxJS Observable'ı output'a çevirme:

```
import { outputFromObservable } from '@angular/core/rxjs-interop';

@Component({...})
export class SearchComponent {
  private term = signal('');

  // Debounced search Observable
  private debouncedTerm$ = toObservable(this.term).pipe(
    debounceTime(300),
    distinctUntilChanged()
  );

  // Observable'ı output'a çevir
  search = outputFromObservable(this.debouncedTerm$);
}
```

DERS 18.3

model() — Two-Way Binding

Hem input hem output — banana-in-a-box [(...)] için.

model() Syntax

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-toggle',
  template: `
    <button (click)="toggle()">
      {{ value() ? 'ON' : 'OFF' }}
    </button>
  `
})
export class ToggleComponent {
  // model() — two-way binding
  value = model.required<boolean>();

  // Veya default value ile
  // value = model(false);

  toggle() {
    this.value.update(v => !v);
  }
}
```

Parent Kullanım

```
// One-way (input gibi)
<app-toggle [value]="isActive()" (valueChange)="onChange($event)" />

// Two-way (banana-in-a-box)
<app-toggle [(value)]="isActive" />
//                    ^^^^^^^^
//                    signal değil, signal kendisi
```

Use Cases

- ✓ Form controls: Input, select, datepicker
- ✓ Toggle / switch components
- ✓ Dropdowns with selected value
- ✓ Editable text fields

- ✓ Slider / range inputs

DERS 18.4

Content Projection — ng-content

Slot-based composition. Parent'tan child'a template enjekte etmek.

Single Slot

```
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content />
    </div>
  `
})
export class CardComponent {}
```

```
<!-- Kullanım -->
<app-card>
  <h2>Card başlığı</h2>
  <p>İçerik</p>
</app-card>
```

Multiple Slots

```
@Component({
  selector: 'app-modal',
  template: `
    <div class="modal">
      <header class="modal-header">
        <ng-content select="[slot=header]" />
      </header>
      <main class="modal-body">
        <ng-content />  <!-- Default slot -->
      </main>
      <footer class="modal-footer">
        <ng-content select="[slot=footer]" />
      </footer>
    </div>
  `
})
export class ModalComponent {}

<!-- Kullanım -->
<app-modal>
  <div slot="header">
    <h2>Modal Başlık</h2>
  </div>

  <p>Ana içerik (default slot)</p>

  <div slot="footer">
    <button>Cancel</button>
    <button>OK</button>
  </div>
</app-modal>
```

□ Conditional Projection

```
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      @if (hasHeader()) {
        <header><ng-content select="[slot=header]" /></header>
      }
      <main><ng-content /></main>
    </div>
  `
})
export class CardComponent {
  // Content child detect
  hasHeader = contentChild<HTMLElement>('header');
}
```

DERS 18.5

viewChild & contentChild

Component'in içindeki child component/element'lere erişim.

🔍 viewChild — Template İçindeki

```
@Component({
  template: `
    <input #emailInput type="email">
    <app-button #saveBtn />
  `
})
export class FormComponent {
  // Element reference
  emailInput = viewChild<ElementRef<HTMLInputElement>>('emailInput');

  // Component reference
  saveBtn = viewChild<ButtonComponent>('saveBtn');

  // Required (Angular 18+)
  requiredEmail = viewChild.required<ElementRef>('emailInput');

  focusEmail() {
    this.emailInput()?.nativeElement.focus();
  }
}
```

📦 contentChild — ng-content İçinden

```
@Component({
  selector: 'app-tabs',
  template: `<div class="tabs"><ng-content /></div>`
})
export class TabsComponent {
  // ng-content içine konulan child component'leri al
  tabs = contentChildren(TabComponent);
  activeTab = computed(() => this.tabs().find(t => t.active()));
}

<!-- Kullanım -->
<app-tabs>
  <app-tab>Tab 1</app-tab>
  <app-tab>Tab 2</app-tab>
</app-tabs>
```

🔍 Çoklu Child — viewChildren

```
@Component({
  template: `
    @for (item of items(); track item.id) {
      <app-item #itemRef [data]="item" />
    }
  `
})
export class ListComponent {
  items = signal<Item[]>([]);

  // Tüm app-item component'lerine erişim
  itemRefs = viewChildren(ItemComponent);

  someMethod() {
    this.itemRefs().forEach(item => item.doSomething());
  }
}
```

DERS 18.6

Sibling Communication

Aynı parent'a sahip iki component konuşturmak.

🔧 Shared Service Pattern

```
@Injectable({ providedIn: 'root' })
export class SharedStateService {
  private _selectedId = signal<string | null>(null);

  readonly selectedId = this._selectedId.asReadonly();

  selectItem(id: string) {
    this._selectedId.set(id);
  }

  clearSelection() {
    this._selectedId.set(null);
  }
}
```

📤 Component A (Source)

```
@Component({
  selector: 'app-list',
  template: `
    @for (item of items(); track item.id) {
      <div (click)="select(item.id)">{{ item.name }}</div>
    }
  `
})
export class ListComponent {
  private shared = inject(SharedStateService);
  items = signal<Item[]>([...]);

  select(id: string) {
    this.shared.selectItem(id);  // Servise gönder
  }
}
```

📥 Component B (Target)

```
@Component({
  selector: 'app-detail',
  template: `
    @if (selectedItem(); as item) {
      <h2>{{ item.name }}</h2>
    } @else {
      <p>Bir item seç</p>
    }
  `
})
export class DetailComponent {
  private shared = inject(SharedStateService);
  private itemService = inject(ItemService);

  // Sibling'in seçtiği ID'yi al
  selectedId = this.shared.selectedId;

  selectedItem = computed(() => {
    const id = this.selectedId();
    return id ? this.itemService.findById(id) : null;
  });
}
```

DERS 18.7

Cross-Tree Communication

Tamamen farklı yerlerdeki component'leri konuşturmak.

🌐 Global State Service

```
@Injectable({ providedIn: 'root' })
export class AppEventBus {
  // Generic event bus
  private events = new Subject<{ type: string; payload: any }>();

  events$ = this.events.asObservable();

  emit(type: string, payload?: any) {
    this.events.next({ type, payload });
  }

  on<T>(type: string) {
    return this.events.pipe(
      filter(e => e.type === type),
      map(e => e.payload as T)
    );
  }
}
```

📤 Component A — Emit

```
@Component({...})
export class HeaderComponent {
  private bus = inject(AppEventBus);

  notifyAll() {
    this.bus.emit('user-action', { action: 'logout' });
  }
}
```

📥 Component B — Listen

```
@Component({...})
export class SidebarComponent {
  private bus = inject(AppEventBus);

  constructor() {
    this.bus.on<{ action: string }>('user-action')
      .pipe(takeUntilDestroyed())
      .subscribe(({ action }) => {
        if (action === 'logout') {
          // Sidebar'da değişiklik yap
        }
      });
  }
}
```

⚠ Anti-Pattern: Event Bus Aşırı Kullanım

> **⚠ Dikkat**
>
> Event bus güçlü ama aşırı kullanma. Component'ler arası açık coupling daha iyi. Event bus sadece genuinely cross-cutting concern'lar için (toast, modal, logout, auth refresh).

□ Decision Tree

Durum	Yaklaşım
Parent → Direct child	input()
Child → Direct parent	output()
Two-way	model()
Template enjekte	ng-content
Sibling	Shared service
Cross-tree (1-1)	Shared service
Cross-tree (broadcast)	Event bus

Modül 18 Özeti

Component communication artık çocuk oyuncağı. Her senaryo için doğru pattern'i biliyorsun.

Neler Öğrendin?

- ✓ input() ve output() modern signal API
- ✓ model() ile two-way binding
- ✓ Content projection (ng-content) — single/multi slot
- ✓ viewChild/contentChild ile element erişimi
- ✓ Sibling communication — shared service
- ✓ Cross-tree — event bus pattern
- ✓ Hangi senaryoda hangi pattern karar verebilmek

Sıradaki Modül

Modül 19 — Internationalization (i18n). Çoklu dil desteği — translate, pluralization, date/number locale.